AMRITA VISHWA VIDYAPEETHAM

22AIE303 Database Management Systems

PROJECT REPORT

Real-Time CDC Pipeline for an E-Commerce Database

ShopDB: PostgreSQL · Debezium · Kafka · Redis

Team Members

Aamith Kishore T J	CB.SC.U4AIE24001
Naveen U	CB.SC.U4AIE24035
Ragav S	CB.SC.U4AIE24041
Padmesh Sivalingam	CB.SC.U4AIE24044

GitHub repository: https://github.com/aamithkishoretj/CDC-pipeline-CC-DBMS-

October 2026

Contents

Contents 1
1. Project Title 3
2. Team Members 4
3. Introduction and Problem Statement 5
3.1 Introduction 5
3.2 Problem Statement 5
3.3 Objectives 5
4. Conceptual and Logical Database Design 6
4.1 Requirements 6
4.2 Conceptual Design (ER Model) 6
4.3 Logical Design (ER to Relational Mapping) 6
4.4 Physical Design 7
5. Normalization and Functional Dependencies 8
5.1 Functional Dependencies 8
5.2 Decomposition from an Unnormalized Order Sheet 8
5.3 Justification 8
6. Advanced SQL Implementation and Query Design 10
6.1 DDL with Constraints 10
6.2 Multi-table Join 10
6.3 GROUP BY, Aggregate Functions and HAVING 10
6.4 Subqueries 11
6.5 Common Table Expressions and Window Functions 12
6.6 Triggers (PL/pgSQL) 12
6.7 Transactions and Concurrency Control 12
6.8 Constraint Enforcement 13
7. ER Diagram and UML Diagram 14
7.1 ER Diagram 14
7.2 UML Use Case Diagram 14
8. Relational Schema 16
9. Database Tables 17
9.1 categories 17
9.2 customers 17
9.3 products 18
9.4 orders 18
9.5 order_items 19

9.6 payments 19
9.7 order_status_history 20
10. Conclusion and Future Scope 21
10.1 Conclusion 21
10.2 Future Scope 21
10.3 Repository 21
References 22

1. Project Title

Real-Time CDC Pipeline for an E-Commerce Database (ShopDB).

A normalized PostgreSQL database for an online shop, together with a Change Data Capture (CDC) pipeline that keeps a fast Redis read model in sync with it. The application writes only to PostgreSQL; Debezium reads PostgreSQL's write-ahead log and streams every committed change through Apache Kafka to a consumer, which builds denormalized documents in Redis that serve all reads.

Table 1.1: Project summary

Item	Value
Course	22AIE303 Database Management Systems
Database	PostgreSQL 15, database `shopdb`
Tables	7: categories, customers, products, orders, order_items, payments, order_status_history
Seed data	6 categories, 24 products, 200 customers, 1,200 orders, 2,400 order items, 1,200 payments, 3,360 status-history rows
Constraints	7 primary keys, 6 foreign keys (3 ON DELETE CASCADE), 3 UNIQUE, 10 CHECK
Triggers	3 (order total, status history, updated_at)
Backend / frontend	Python Flask (Write API, Read API) / HTML, CSS, JavaScript served by nginx
Repository	https://github.com/aamithkishoretj/CDC-pipeline-CC-DBMS-

2. Team Members

Table 2.1: Team

#	Name	Roll number
1	Aamith Kishore T J	CB.SC.U4AIE24001
2	Naveen U	CB.SC.U4AIE24035
3	Ragav S	CB.SC.U4AIE24041
4	Padmesh Sivalingam	CB.SC.U4AIE24044

3. Introduction and Problem Statement

3.1 Introduction

Online shops perform a steady stream of writes (orders, payments, status changes) and a much larger volume of reads (order tracking, order history, product pages). A relational database such as PostgreSQL gives strong guarantees for the writes through ACID transactions, integrity constraints and triggers. Serving every read from the same database, however, makes the reads compete with the writes for CPU, locks and I/O.

This project designs the transactional database of an online shop from requirements to a normalized schema, implements it in PostgreSQL with constraints, triggers, transactions and advanced SQL, and adds a read model in Redis that is kept consistent with the database through Change Data Capture.

3.2 Problem Statement

- **Heavy-read bottleneck.** Showing one complete order requires a join across six tables. Measured on this project, PostgreSQL read latency almost doubles when writes run concurrently (1.24 ms → 2.20 ms per order).
- **Dual-write inconsistency.** The usual remedy writes each change to the database and then to a cache. If the application fails between the two writes, the cache is permanently inconsistent with the database and nothing repairs it.

The system must therefore let the application write to exactly one database, serve reads without loading that database, and guarantee that the fast read copy always converges to the database.

3.3 Objectives

1. Model the shop with an ER diagram and map it to a relational schema (CO2).
2. Normalize the schema to 3NF/BCNF and justify it with functional dependencies (CO3).
3. Implement constraints, triggers and advanced SQL queries (CO1).
4. Guarantee ACID transactions and concurrency control for order processing (CO4).
5. Use the right database for each job: PostgreSQL for transactions, Kafka for the change log and Redis for reads (CO5).
6. Build a backend and frontend that demonstrate every database operation.

4. Conceptual and Logical Database Design

4.1 Requirements

- A **customer** has a name, a unique email, a phone number, an address and a city, and can place many orders.
- A **product** belongs to exactly one **category** and has a price and a stock level.
- An **order** belongs to one customer, has a date, a status (pending, processing, shipped, delivered) and a total amount, and contains one or more products with a quantity and the price paid.
- Each order is paid by a **payment** (card, UPI, net banking or cash on delivery).
- Every status change of an order must be recorded with its time (audit trail).

4.2 Conceptual Design (ER Model)

Table 4.1: Entities, attributes and keys

Entity	Attributes	Key
Category	category_id, name, description	category_id (name is an alternate key)
Customer	customer_id, name, email, phone, address, city, created_at	customer_id (email is an alternate key)
Product	product_id, name, description, price, stock, updated_at	product_id
Order	order_id, order_date, status, total_amount (derived)	order_id
Payment	payment_id, method, amount, status, paid_at	payment_id
Order Status History	history_id, old_status, new_status, changed_at	history_id

Table 4.2: Relationships and cardinalities

Relationship	Entities	Cardinality	Participation
classifies	Category – Product	1 : N	every product has one category
places	Customer – Order	1 : N	every order has one customer
contains	Order – Product (attributes: quantity, unit_price)	M : N	an order has at least one product
paid by	Order – Payment	1 : N	payment depends on its order
logged in	Order – Order Status History	1 : N	history depends on its order

4.3 Logical Design (ER to Relational Mapping)

- Each entity becomes a relation with a surrogate `SERIAL` primary key.
- Each 1 : N relationship becomes a `NOT NULL` foreign key on the N side (`products.category_id`, `orders.customer_id`, `payments.order_id`, `order_status_history.order_id`).

- The M : N relationship *contains* becomes the relation `order_items(order_item_id, order_id, product_id, quantity, unit_price)` with `UNIQUE (order_id, product_id)`; its relationship attributes become columns.
- Children of an order (items, payments, history) are removed with `ON DELETE CASCADE`; products and customers are protected (`NO ACTION`) because orders refer to them.
- Domain rules become `CHECK` constraints; money is stored as `DECIMAL(10,2)`.

4.4 Physical Design

- B-tree indexes on every foreign-key column and on `orders.status`.
- Three triggers keep derived and audit data correct inside the same transaction (section 6.6).
- Least-privilege roles: `app_writer` (DML only, used by the application) and `debezium` (SELECT + REPLICATION).
- `REPLICA IDENTITY FULL` and the publication `shopdb_publication` allow Debezium to stream complete before and after images of each row.

5. Normalization and Functional Dependencies

5.1 Functional Dependencies

Table 5.1: Functional dependencies and candidate keys

Relation	Functional dependencies	Candidate keys	NF
categories	category_id → name, description; name → category_id, description	{category_id}, {name}	BCNF
customers	customer_id → name, email, phone, address, city, created_at; email → customer_id, name, phone, address, city, created_at	{customer_id}, {email}	BCNF
products	product_id → category_id, name, description, price, stock, updated_at	{product_id}	BCNF
orders	order_id → customer_id, order_date, status, total_amount	{order_id}	BCNF
order_items	order_item_id → order_id, product_id, quantity, unit_price; (order_id, product_id) → order_item_id, quantity, unit_price	{order_item_id}, {order_id, product_id}	BCNF
payments	payment_id → order_id, method, amount, status, paid_at	{payment_id}	BCNF
order_status_history	history_id → order_id, old_status, new_status, changed_at	{history_id}	BCNF

5.2 Decomposition from an Unnormalized Order Sheet

Without normalization, an order would be recorded as a single sheet:

```
OrderSheet(order_id, order_date, customer_name, email, city,
           {product_name, category, quantity, price}, pay_method, status)
```

Table 5.2: Normalization steps

Step	Result	Problem removed
UNF → 1NF	OrderLine(**order_id, product_id**, order_date, customer_id, customer_name, email, city, product_name, category, quantity, unit_price, pay_method, status)	Repeating group of products; every value is now atomic
1NF → 2NF	Orders(**order_id**, order_date, customer_id, customer_name, email, city, pay_method, status); Products(**product_id**, product_name, category); OrderItems(**order_id, product_id**, quantity, unit_price)	Partial dependencies: order_id → order data and product_id → product data depended on only part of the key
2NF → 3NF	customers, orders, payments, order_status_history, categories, products, order_items	Transitive dependencies: order_id → customer_id → name, email, city; product_id → category_id → category name; payment and status history separated
3NF → BCNF	Already satisfied	Every determinant (customer_id, email, category_id, name, order_id, …) is a candidate key

5.3 Justification

- **1NF:** every attribute holds one atomic value; the products of an order are rows of `order_items`.

- **2NF:** no non-key attribute depends on part of a key. The only composite candidate key is (order_id, product_id) in `order_items`, and quantity and unit_price depend on the whole pair.
- **3NF:** no transitive dependency. Customer details exist only in `customers` and category names only in `categories`.
- **BCNF:** for every non-trivial dependency X → Y, X is a superkey, including the alternate keys `customers.email` and `categories.name`.
- `order_items.unit_price` records the price **at the time of purchase**. A later change of `products.price` must not alter historical orders, so it is a fact of the order line and not a transitive dependency.
- `orders.total_amount` is a derived attribute. It is maintained by a trigger inside the same transaction, so it cannot become inconsistent (controlled redundancy).

Anomalies removed: an email change updates one row (update anomaly); a product can be inserted before anyone orders it (insertion anomaly); deleting the last order of a product does not delete the product (deletion anomaly).

6. Advanced SQL Implementation and Query Design

All queries are documented in `database/queries.sql` and their complete outputs on a freshly seeded database are recorded in `database/sample_outputs.txt`. The outputs below are from that run.

6.1 DDL with Constraints

```
CREATE TABLE orders (
    order_id      SERIAL PRIMARY KEY,
    customer_id   INT NOT NULL REFERENCES customers(customer_id),
    order_date    TIMESTAMP NOT NULL DEFAULT NOW(),
    status        VARCHAR(20) NOT NULL
                  CHECK (status IN ('pending','processing','shipped','delivered')),
    total_amount  DECIMAL(10,2) NOT NULL CHECK (total_amount >= 0)
);
CREATE TABLE order_items (
    order_item_id SERIAL PRIMARY KEY,
    order_id      INT NOT NULL REFERENCES orders(order_id) ON DELETE CASCADE,
    product_id    INT NOT NULL REFERENCES products(product_id),
    quantity      INT NOT NULL CHECK (quantity > 0),
    unit_price    DECIMAL(10,2) NOT NULL CHECK (unit_price >= 0),
    UNIQUE (order_id, product_id)
);
```

6.2 Multi-table Join

Rebuilds order 1 from its normalized parts by joining five tables.

```
SELECT o.order_id, c.name AS customer, p.name AS product, cat.name AS category,
       oi.quantity, oi.unit_price, oi.quantity * oi.unit_price AS line_total
FROM orders o
JOIN customers   c   ON c.customer_id   = o.customer_id
JOIN order_items oi  ON oi.order_id     = o.order_id
JOIN products    p   ON p.product_id    = oi.product_id
JOIN categories  cat ON cat.category_id = p.category_id
WHERE o.order_id = 1;
```

Table 6.1: Output of the join

order_id	customer	product	category	qty	unit_price	line_total
1	Lakshmi Rao	Wireless Earbuds	Electronics	1	2499.00	2499.00
1	Lakshmi Rao	Water Bottle	Home	2	599.00	1198.00

6.3 GROUP BY, Aggregate Functions and HAVING

```
SELECT cat.name AS category, COUNT(DISTINCT oi.order_id) AS orders,
       SUM(oi.quantity) AS units, SUM(oi.quantity * oi.unit_price) AS revenue
FROM order_items oi
JOIN products p     ON p.product_id = oi.product_id
JOIN categories cat ON cat.category_id = p.category_id
GROUP BY cat.name
ORDER BY revenue DESC;
```

Table 6.2: Revenue by category

category	orders	units	revenue (₹)
Computers	515	1,218	58,31,182.00
Electronics	335	713	20,48,487.00
Home	388	841	16,53,959.00
Fashion	289	618	14,45,082.00
Sports	258	524	13,39,376.00
Books	391	876	9,45,793.00

HAVING filters groups after aggregation, which WHERE cannot do:

```
SELECT method, COUNT(*) AS payments, SUM(amount) AS total_amount
FROM payments GROUP BY method
HAVING SUM(amount) > 3300000
ORDER BY total_amount DESC;          -- cod 36,94,552.00 ; netbanking 34,04,190.00
```

6.4 Subqueries

Customers whose total spending is above the average customer's spending (nested subquery inside HAVING):

```
SELECT c.customer_id, c.name, c.city, SUM(o.total_amount) AS total_spent
FROM customers c JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.name, c.city
HAVING SUM(o.total_amount) > (
    SELECT AVG(t.spent)
    FROM (SELECT SUM(total_amount) AS spent FROM orders GROUP BY customer_id) t)
ORDER BY total_spent DESC LIMIT 5;
```

Table 6.3: Top customers above average

customer_id	name	city	total_spent (₹)
199	Pooja Kumar	Ahmedabad	2,40,492.00
22	Nikhil Reddy	Chennai	1,88,252.00
161	Divya Nambiar	Mumbai	1,87,063.00
47	Anjali Pillai	Kochi	1,85,533.00
159	Anjali Rao	Ahmedabad	1,64,000.00

Table 6.4: Other subqueries

Technique	Question	Result
Scalar subquery	How many orders are above the average order value?	412 orders (average ₹11,053.23)
IN subquery	How many customers bought from Electronics?	168
Correlated NOT EXISTS	How many customers have no delivered order?	19
Correlated subquery	Which products cost more than their category's average?	8 products

6.5 Common Table Expressions and Window Functions

```
SELECT category, product, units FROM (
  SELECT cat.name AS category, p.name AS product, SUM(oi.quantity) AS units,
         RANK() OVER (PARTITION BY cat.name ORDER BY SUM(oi.quantity) DESC) AS rnk
  FROM order_items oi
  JOIN products p     ON p.product_id = oi.product_id
  JOIN categories cat ON cat.category_id = p.category_id
  GROUP BY cat.name, p.name) ranked
WHERE rnk = 1 ORDER BY units DESC;
```

Table 6.5: Best-selling product per category

category	product	units
Fashion	Denim Jacket	252
Books	The Pragmatic Programmer	234
Home	Electric Kettle	229
Computers	Portable SSD 1TB	227
Electronics	Wireless Earbuds	204
Sports	Dumbbell Set	200

`queries.sql` also contains a CTE with DENSE_RANK (top customer in each city), a running total with `SUM() OVER (ORDER BY …)` and the view `order_summary`.

6.6 Triggers (PL/pgSQL)

```
CREATE FUNCTION fn_recalc_order_total() RETURNS TRIGGER AS $$
DECLARE oid INT := COALESCE(NEW.order_id, OLD.order_id);
BEGIN
    UPDATE orders SET total_amount = COALESCE(
        (SELECT SUM(quantity * unit_price) FROM order_items WHERE order_id = oid), 0)
     WHERE order_id = oid;
    RETURN NULL;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_order_items_total
AFTER INSERT OR UPDATE OR DELETE ON order_items
FOR EACH ROW EXECUTE FUNCTION fn_recalc_order_total();
```

Table 6.6: Triggers

Trigger	Table	Timing / events	Purpose
trg_order_items_total	order_items	AFTER INSERT, UPDATE, DELETE	Keeps orders.total_amount = Σ quantity × unit_price
trg_orders_status_history	orders	AFTER INSERT, UPDATE OF status	Writes an audit row for every status change
trg_products_touch	products	BEFORE UPDATE	Maintains products.updated_at

6.7 Transactions and Concurrency Control

Placing an order is a single ACID transaction. The product rows are locked with `SELECT … FOR UPDATE` in a fixed order (by product_id), so two concurrent orders can never deadlock; the locks are held until COMMIT (strict two-phase locking). If the stock is insufficient or any constraint fails, the whole transaction is rolled back.

```
BEGIN;
  SELECT product_id, price, stock FROM products
   WHERE product_id = ANY(%s) ORDER BY product_id FOR UPDATE;
  INSERT INTO orders (customer_id, status, total_amount) VALUES (%s, 'pending', 0);
  INSERT INTO order_items (order_id, product_id, quantity, unit_price) VALUES (...);
  UPDATE products SET stock = stock - %s WHERE product_id = %s;   -- CHECK stock >= 0
  INSERT INTO payments (order_id, method, amount, status) VALUES (...);
COMMIT;
```

Demonstration from `queries.sql`: inside a transaction, an order for one Wireless Earbuds (₹2,499) and one Smartwatch (₹4,999) is inserted. The trigger sets its total to **₹7,498.00** and writes the history row (NULL → pending). After **ROLLBACK** the database still holds exactly 1,200 orders (atomicity).

The backend uses a `psycopg2` connection pool and parameterized queries (`%s` placeholders), so user input is never concatenated into SQL and SQL injection is not possible.

6.8 Constraint Enforcement

Table 6.7: Statements rejected by the DBMS

Statement	Rejected by
UPDATE products SET stock = -1 WHERE product_id = 1;	CHECK (stock >= 0)
INSERT INTO orders (customer_id, status, total_amount) VALUES (99999, 'pending', 0);	FOREIGN KEY orders_customer_id_fkey
UPDATE orders SET status = 'lost' WHERE order_id = 1;	CHECK (status IN (...))
INSERT INTO categories (name) VALUES ('Books');	UNIQUE (name)

7. ER Diagram and UML Diagram

7.1 ER Diagram

Figure 7.1 shows the seven entities with their attributes, primary keys (PK), foreign keys (FK), unique attributes (UQ) and relationship cardinalities in crow's-foot notation.



Figure 7.1: ER diagram of shopdb

7.2 UML Use Case Diagram

Figure 7.2 shows the use cases of the system. The **Customer** browses products, places and tracks orders and views the order history; placing an order includes making the payment and updating the stock. The **Shop Admin** updates order status (which includes recording the status history), deletes orders with restocking, manages products, stock and customers, views sales reports and monitors the pipeline. The **CDC Pipeline** is a system actor that keeps the Redis read model in sync.

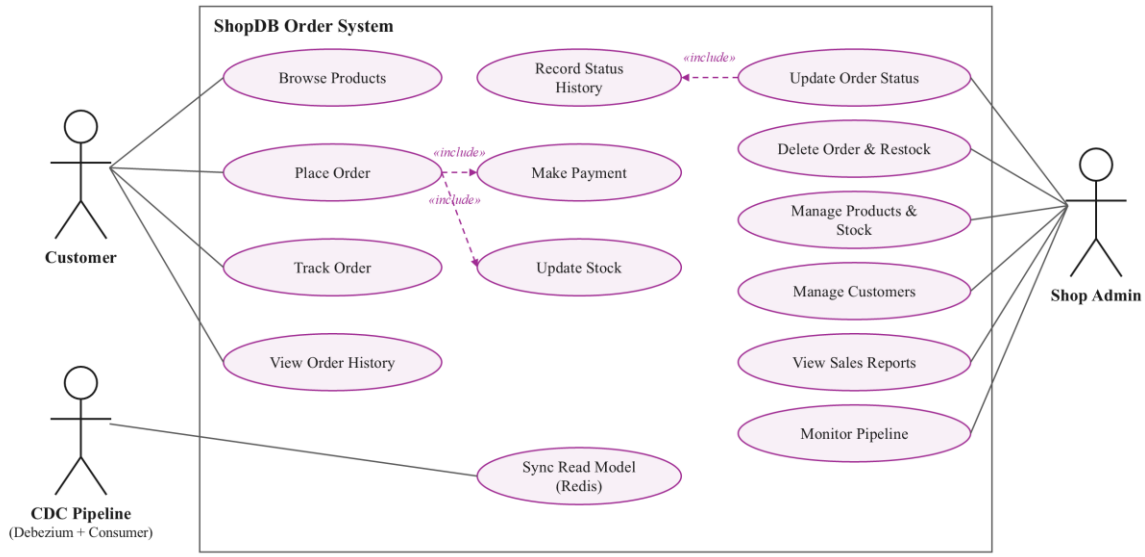


Figure 7.2: UML use case diagram of the ShopDB order system

8. Relational Schema

In the notation below, the first attribute of each relation is its primary key and → marks a foreign key and the relation it references.

```
categories(category_id, name, description)
customers(customer_id, name, email, phone, address, city, created_at)
products(product_id, category_id → categories, name, description, price, stock, updated_at)
orders(order_id, customer_id → customers, order_date, status, total_amount)
order_items(order_item_id, order_id → orders, product_id → products, quantity, unit_price)
payments(payment_id, order_id → orders, method, amount, status, paid_at)
order_status_history(history_id, order_id → orders, old_status, new_status, changed_at)
```

Table 8.1: Relational schema with keys and constraints

Relation	Primary key	Foreign keys	Other constraints
categories	**category_id**	-	name UNIQUE, NOT NULL
customers	**customer_id**	-	email UNIQUE + CHECK (format); name, address, city NOT NULL
products	**product_id**	category_id → categories (NO ACTION)	CHECK price > 0, stock ≥ 0
orders	**order_id**	customer_id → customers (NO ACTION)	CHECK status IN (pending, processing, shipped, delivered); total ≥ 0
order_items	**order_item_id**	order_id → orders (CASCADE); product_id → products (NO ACTION)	CHECK quantity > 0, unit_price ≥ 0; UNIQUE (order_id, product_id)
payments	**payment_id**	order_id → orders (CASCADE)	CHECK method IN (card, upi, netbanking, cod); amount ≥ 0; status IN (pending, completed, failed, refunded)
order_status_history	**history_id**	order_id → orders (CASCADE)	new_status NOT NULL; written only by a trigger

Table 8.2: Indexes

Index	Column(s)
idx_products_category	products(category_id)
idx_orders_customer	orders(customer_id)
idx_orders_status	orders(status)
idx_order_items_order	order_items(order_id)
idx_order_items_product	order_items(product_id)
idx_payments_order	payments(order_id)
idx_history_order	order_status_history(order_id)

9. Database Tables

Each table is listed with its structure (attributes, data types, keys and constraints) and sample records taken from a freshly seeded database. The data are created by `database/02_seed.sql` with a fixed random seed, so they are identical on every load.

Table 9.1: Row counts

Table	Rows
categories	6
products	24
customers	200
orders	1,200
order_items	2,400
payments	1,200
order_status_history	3,360

9.1 categories

Table 9.2: Structure of categories

Attribute	Data type	Key / constraint
category_id	SERIAL	PRIMARY KEY
name	VARCHAR(60)	NOT NULL, UNIQUE
description	TEXT	

Table 9.3: Sample records of categories

category_id	name	description
1	Electronics	Phones, audio and gadgets
2	Computers	Laptops, peripherals and accessories
3	Home	Home and kitchen essentials
4	Books	Fiction, non-fiction and textbooks

9.2 customers

Table 9.4: Structure of customers

Attribute	Data type	Key / constraint
customer_id	SERIAL	PRIMARY KEY
name	VARCHAR(120)	NOT NULL
email	VARCHAR(160)	NOT NULL, UNIQUE, CHECK (email LIKE '%_@_%._%')
phone	VARCHAR(20)	
address	TEXT	NOT NULL
city	VARCHAR(60)	NOT NULL
created_at	TIMESTAMP	NOT NULL DEFAULT NOW()

Table 9.5: Sample records of customers

customer_id	name	email	phone	address	city
1	Aditi Rao	aditi.rao@example.com	9800000001	12 MG Road	Bengaluru
2	Karan Mehta	karan.mehta@example.com	9800000002	45 Linking Road	Mumbai
3	Sara Iyer	sara.iyer@example.com	9800000003	8 Anna Salai	Chennai
4	Rahul Nair	rahul.nair@example.com	9800000004	21 MG Road	Kochi

9.3 products

Table 9.6: Structure of products

Attribute	Data type	Key / constraint
product_id	SERIAL	PRIMARY KEY
category_id	INT	NOT NULL, FK → categories
name	VARCHAR(160)	NOT NULL
description	TEXT	
price	DECIMAL(10,2)	NOT NULL, CHECK (price > 0)
stock	INT	NOT NULL DEFAULT 0, CHECK (stock >= 0)
updated_at	TIMESTAMP	NOT NULL DEFAULT NOW(); trigger

Table 9.7: Sample records of products

product_id	category_id	name	description	price	stock
1	1	Wireless Earbuds	Bluetooth 5.3, 30h battery	2499.00	320
2	1	Smartwatch	AMOLED display, heart-rate sensor	4999.00	150
3	1	Bluetooth Speaker	Portable, waterproof	1899.00	260
4	1	Power Bank 20000mAh	Fast charging, USB-C	1599.00	400

9.4 orders

Table 9.8: Structure of orders

Attribute	Data type	Key / constraint
order_id	SERIAL	PRIMARY KEY
customer_id	INT	NOT NULL, FK → customers
order_date	TIMESTAMP	NOT NULL DEFAULT NOW()
status	VARCHAR(20)	NOT NULL, CHECK IN (pending, processing, shipped, delivered)
total_amount	DECIMAL(10,2)	NOT NULL, CHECK (>= 0); maintained by trigger

Table 9.9: Sample records of orders

order_id	customer_id	order_date	status	total_amount
1	149	2026-09-20 11:32	delivered	3697.00
2	188	2026-10-17 07:32	delivered	34796.00
3	19	2026-10-03 20:32	delivered	6897.00
4	99	2026-09-18 01:42	shipped	1498.00

9.5 order_items

Table 9.10: Structure of order_items

Attribute	Data type	Key / constraint
order_item_id	SERIAL	PRIMARY KEY
order_id	INT	NOT NULL, FK → orders ON DELETE CASCADE
product_id	INT	NOT NULL, FK → products
quantity	INT	NOT NULL, CHECK (quantity > 0)
unit_price	DECIMAL(10,2)	NOT NULL, CHECK (unit_price >= 0)
(order_id, product_id)		UNIQUE

Table 9.11: Sample records of order_items

order_item_id	order_id	product_id	quantity	unit_price
1	1	1	1	2499.00
2	1	14	2	599.00
3	2	8	2	15999.00
4	2	9	1	999.00

9.6 payments

Table 9.12: Structure of payments

Attribute	Data type	Key / constraint
payment_id	SERIAL	PRIMARY KEY
order_id	INT	NOT NULL, FK → orders ON DELETE CASCADE
method	VARCHAR(20)	NOT NULL, CHECK IN (card, upi, netbanking, cod)
amount	DECIMAL(10,2)	NOT NULL, CHECK (amount >= 0)
status	VARCHAR(20)	NOT NULL DEFAULT 'pending', CHECK IN (pending, completed, failed, refunded)
paid_at	TIMESTAMP	

Table 9.13: Sample records of payments

payment_id	order_id	method	amount	status	paid_at
1	8	card	23393.00	completed	2026-10-05 15:20
2	9	upi	2499.00	completed	2026-08-05 06:35
3	6	netbanking	4497.00	completed	2026-07-27 20:32
4	7	cod	5847.00	completed	2026-08-09 15:12

9.7 order_status_history

Table 9.14: Structure of order_status_history

Attribute	Data type	Key / constraint
history_id	SERIAL	PRIMARY KEY
order_id	INT	NOT NULL, FK → orders ON DELETE CASCADE
old_status	VARCHAR(20)	NULL for the first status
new_status	VARCHAR(20)	NOT NULL
changed_at	TIMESTAMP	NOT NULL DEFAULT NOW()

Table 9.15: Sample records of order_status_history

history_id	order_id	old_status	new_status	changed_at
1	1	NULL	pending	2026-09-20 11:32
2	2	NULL	pending	2026-10-17 07:32
3	3	NULL	pending	2026-10-03 20:32
4	4	NULL	pending	2026-09-18 01:42

10. Conclusion and Future Scope

10.1 Conclusion

The project designed the database of an online shop from requirements through an ER model and a relational mapping to a schema of seven relations that is in Boyce-Codd normal form. PostgreSQL itself enforces every rule: primary and foreign keys with cascading deletes, unique and check constraints, triggers for the order total and the status audit trail, and least-privilege roles. Every business operation is a single ACID transaction with row-level locking in a fixed order, which prevents both overselling and deadlocks.

Advanced SQL (multi-table joins, grouping with HAVING, scalar, IN and correlated subqueries, CTEs, window functions and views) answers the shop's reporting questions directly from the normalized data. Change Data Capture keeps a denormalized Redis read model in sync with the database through Kafka, which removes the dual-write problem and serves a complete order 11 times faster than the six-table join. All fifteen automated end-to-end tests pass.

10.2 Future Scope

- Table partitioning of `orders` by date and materialized views for monthly sales reports.
- User authentication with row-level security, so each customer can read only their own orders.
- Additional entities such as product reviews, wish lists, coupons and shipments.
- Use the change stream for analytics and machine learning, for example demand forecasting and product recommendations.
- Managed database services and automated backups with point-in-time recovery.

10.3 Repository

Source code, SQL scripts, documentation, tests and screenshots: **https://github.com/aamithkishoretj/CDC-pipeline-CC-DBMS-**

Table 10.1: Repository contents

Folder	Contents
database/	01_schema.sql (DDL), 02_seed.sql (data), 03_roles.sql (roles), queries.sql (documented queries), sample_outputs.txt
services/	Flask Write API (PostgreSQL) and Read API (Redis)
frontend/	Website (HTML, CSS, JavaScript) and nginx configuration
consumer/, debezium/	CDC projector and connector configuration
tests/	End-to-end and failover test cases
docs/	ER diagram, UML diagram, screenshots

References

[1] A. Silberschatz, H. F. Korth and S. Sudarshan, Database System Concepts, 6th ed. McGraw-Hill, 2011.

[2] H. Garcia-Molina, J. D. Ullman and J. Widom, Database Systems: The Complete Book, 2nd ed. Pearson, 2011.

[3] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 5th ed. Addison-Wesley.

[4] PostgreSQL Global Development Group, PostgreSQL 15 Documentation. https://www.postgresql.org/docs/15/

[5] Debezium Project, Debezium Connector for PostgreSQL. https://debezium.io/documentation/

[6] Redis Ltd., Redis Documentation. https://redis.io/docs/